

A*-plus path planning algorithm for large-scale grid maps

Chaosheng Zhang, Qingsen Hu^{*}, Jiangang Tu, Yu Dong, Yizhong Tan, Yufeng He

Professional Education and Field Training Base, Army Engineering University of PLA, Jiangsu, 221004, Xuzhou, China

ARTICLE INFO

Keywords:

Path planning
A* algorithm
Grid map
Algorithm optimization

ABSTRACT

The A* algorithm is widely recognized as one of the most effective solutions for grid-based path planning. However, when it comes to complex obstacle fields, its performance significantly degrades on large-scale grid maps exceeding $10,000 \times 10,000$ in size, where computational complexity and memory usage increase drastically. To address these challenges and accelerate the path planning tasks, we propose the **A*-plus algorithm**, which introduces a node compression strategy by aggregating 2×2 grid cells into single nodes, thereby substantially reducing the search space. Derived entirely from the A* framework, A*-plus is fully compatible with existing optimization techniques designed for A*, such as priority queues and heuristic functions. Experimental evaluations on $10,000 \times 10,000$ grid maps with varying obstacle densities show that A*-plus achieves a 93 % reduction in search time and a 60 % decrease in memory usage compared to the traditional A* algorithm, without compromising path optimality. These results demonstrate the effectiveness of A*-plus in enabling real-time path planning in expansive grid environments.

1. Introduction

Path planning plays a vital role in autonomous navigation, robotics, and intelligent transportation systems. With the increasing complexity of real-world environments and the expansion of map scales, efficient and reliable path planning algorithms have become essential for ensuring real-time decision-making and safety in large-scale navigation tasks.

The classical A* algorithm, based on heuristic search, has long been the foundation of optimal path planning. However, as map resolution and environmental complexity grow, the computational cost of A* increases exponentially. This leads to performance degradation, particularly in large-scale or dynamic environments.

Numerous variants of A* have been proposed to improve computational efficiency, such as Jump Point Search (JPS) [1], Bidirectional A* [2], Hierarchical Path-Finding A* (HPA*) [3], and more recent hybrid approaches combining heuristic refinement and parallel computing [4–6]. Although these methods reduce redundant searches and improve scalability, they still face difficulties in balancing efficiency, memory consumption, and global optimality in ultra-large-scale environments. Especially when performing path planning multiple times within a very large static map, the repeated calculations will impose a huge computational load.

Motivated by these limitations, this study proposes the A-plus algorithm^{*}, which integrates map compression preprocessing, adaptive heuristic optimization, and search-space pruning to enhance path planning efficiency under large-scale and complex environments. Unlike the traditional A* method, A*-plus adopts a mechanism of compressing map nodes in a 2×2 pattern to reduce the number of searches, thereby demonstrating superior performance on large-scale maps. This improved strategy is at a very fundamental level and does not conflict with the mainstream methods currently in use, such as JPS and Bidirectional A*. It can be combined with these methods to further enhance the speed of the search process.

2. Literature review

The A* algorithm, introduced by Hart et al., in 1968 [7], is a path planning algorithm that combines the principles of Dijkstra's algorithm with heuristic search. Its core mechanism relies on an evaluation function, $f(n) = g(n) + h(n)$, which ensures both optimality and efficient pathfinding. However, when applied to large-scale grid maps, the search space and computational complexity of A* increase rapidly, resulting in a sharp decline in efficiency [8].

Grid maps are widely used in path planning research due to their simple structure and ease of implementation. In such maps, path

^{*} Corresponding author.

E-mail addresses: cs116@163.com (C. Zhang), xuzhouhuqingsen@163.com (Q. Hu), 785100639@qq.com (J. Tu), 1251777548@qq.com (Y. Dong), tanyizhong@aeu.edu.cn (Y. Tan), victor02468@126.com (Y. He).

<https://doi.org/10.1016/j.array.2025.100593>

Received 13 June 2025; Received in revised form 21 October 2025; Accepted 19 November 2025

Available online 1 December 2025

2590-0056/© 2025 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

planning algorithms aim to determine the optimal route from a start point to a destination by navigating through discrete grid cells [11]. However, as the scale of the map increases, the performance of traditional algorithms degrades significantly in terms of efficiency.

To enhance the performance of the A* algorithm, various variants have been proposed. Table 1 presents six major improvement methods along with their principles.

We need to admit that these algorithms are all methods for speeding up path planning for large-scale maps, but each of them has some drawbacks, as shown in Table 2.

Additional improvements include enhanced heuristic functions, search space pruning, and path smoothing techniques [9]. For instance, the EBS-A* algorithm incorporates extended distance metrics, bidirectional search, and heuristic optimization to improve both efficiency and robustness [10]. While these methods have enhanced the A* algorithm to some extent, they still face challenges in handling ultra-large maps, particularly with respect to real-time performance.

Apart from heuristic search methods, numerous studies have also employed meta-heuristic approaches to enhance or supplement path planning based on A*. Meta-heuristic algorithms such as genetic algorithms (GA) [14], particle swarm optimization (PSO) [15], ant colony optimization (ACO) [16], and simulated annealing (SA) [17] have been used to address the limitations of A* in complex or large-scale environments. Recent research trends have emphasized hybrid methods that combine meta-heuristic global exploration with the efficiency of A* variants to achieve balanced performance in path planning. However, in large-scale maps and tasks that require multiple path plans, the performance of these methods is still not very satisfactory.

To address this challenge, the A*-plus algorithm proposed in this study leverages map compression preprocessing, heuristic function optimization, and search space pruning. These strategies are designed to enhance both the computational efficiency and the quality of path planning on large-scale grid maps.

3. Algorithm design

3.1. Overview of the A*-plus algorithm

The A*-plus algorithm is an enhanced version of A* designed specifically for large-scale maps, typically containing over 100 million nodes. As illustrated in Fig. 1, the core idea is to aggregate every 2×2 block of grid cells into a single compressed node. By redefining the adjacency relationships between these compressed nodes, the total number of nodes is reduced to one-fourth of the original, effectively preventing

Table 1
Six variants that improve upon the A* algorithm.

Algorithm Name	Principle
Jump Point Search(JPS) [1]	Accelerates A* on uniform-cost grids by skipping symmetric and redundant nodes through directional “jumping,” reducing the number of expansions dramatically.
Bidirectional A* [2]	Performs simultaneous forward and backward searches from the start and goal, meeting in the middle to shorten search depth and computation time.
Hierarchical Path-Finding A* (HPA*) [3]	Divides the map into clusters, builds an abstract graph of entrances, and plans first on the high-level graph before refining the path locally.
A*, Landmarks, Triangle inequality(ALT) [4]	Enhances A* with precomputed distances to selected landmarks and uses the triangle inequality to derive tighter admissible heuristics.
Multi-Resolution A*(MRA*) [5]	Combines coarse and fine search resolutions, using a low-resolution grid for global exploration and high-resolution refinement near the final path.
Weighted-A* [6]	Speeds up A* by inflating the heuristic with a weight factor greater than one, producing faster but potentially suboptimal paths.

Table 2

The drawbacks of six variants that improve the A* algorithm.

Algorithm Name	Drawback
Jump Point Search(JPS) [1]	Its efficiency drops significantly on non-uniform cost maps or highly cluttered environments where symmetry pruning is less effective.
Bidirectional A* [2]	It can be difficult to determine an effective meeting point and maintain consistent heuristics between the two search fronts.
Hierarchical Path-Finding A* (HPA*) [3]	The path optimality depends heavily on the quality of the abstraction, and building or updating the hierarchy adds preprocessing overhead.
A*, Landmarks, Triangle inequality(ALT) [4]	Requires costly preprocessing and large memory to store landmark distances, making it less suitable for dynamic or frequently changing maps.
Multi-Resolution A*(MRA*) [5]	Managing transitions between resolutions and maintaining path consistency can be complex and computationally expensive.
Weighted-A* [6]	The use of an inflated heuristic sacrifices optimality guarantees and may produce paths that are longer than the true shortest route.

the exponential growth of search time.

However, compressing four grid cells into one introduces a fundamental challenge: how to determine the traversability between adjacent compressed nodes. This issue is addressed in detail in Section 3.2.

3.2. Core principles of the A*-plus algorithm

The core idea of the A*-plus algorithm is to accelerate pathfinding on large-scale maps by compressing every 2×2 group of grid cells into a single node. However, for these compressed nodes, the traversability between neighboring blocks must be reconsidered. Unlike the traditional A* algorithm, which allows bidirectional movement as long as adjacent nodes are non-obstructed, A*-plus requires a new set of traversal rules due to the nature of node aggregation.

The A*-plus algorithm introduces several key rules for determining valid traversal.

3.2.1. Mapping start and goal positions to compressed nodes

For a $10,000 \times 10,000$ grid map, if the start point is located at (1, 2) and the goal at (10,000, 10,000), their respective compressed node coordinates become (1, 1) and (5000, 5000) after node aggregation.

3.2.2. Directional search constraints using a 4×4 boolean table

In traditional A*, direction tracking is unnecessary—search proceeds by simply checking whether neighboring cells are obstacle-free. As shown in Fig. 2(a), search is performed freely in four directions. However, in A*-plus, as shown in Fig. 2(b), each red cell block represents a compressed node and orange blocks indicate its neighbors. Traversability is determined by whether any touching border cells between the red and orange blocks are non-obstructed.

A special case is illustrated in Fig. 2(c), where diagonal obstacles impose directional constraints. For example, if the previous move was to the right, then subsequent moves to the right or upward may be blocked. Similarly, if the previous step was downward, movement to the left or down may also be restricted.

To enforce these rules, a 4×4 boolean matrix is used to represent directional access between nodes. The rows represent the direction of the previous move (up, down, left, right), and the columns represent the next move. A value of ‘1’ indicates the move is allowed, and ‘0’ indicates it is not. This matrix is shown in Fig. 3(a).

Note that by default, round-trip movement is not allowed here. For example, when it was moving upwards last time, the current node is not allowed to search downwards because searching for nodes in this way would be repetitive. In the A*-plus algorithm, a ‘close list’ will also be set to record the searched nodes and prevent duplicate searches. Recording the bool table in this way can further simplify the search method. Fig. 3

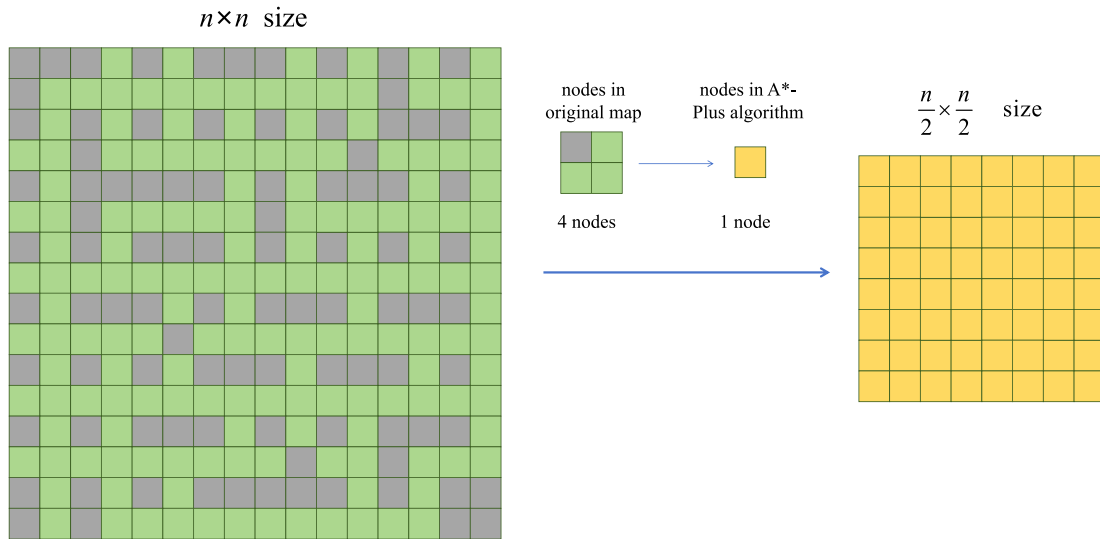


Fig. 1. Core concept of the A*-plus algorithm.

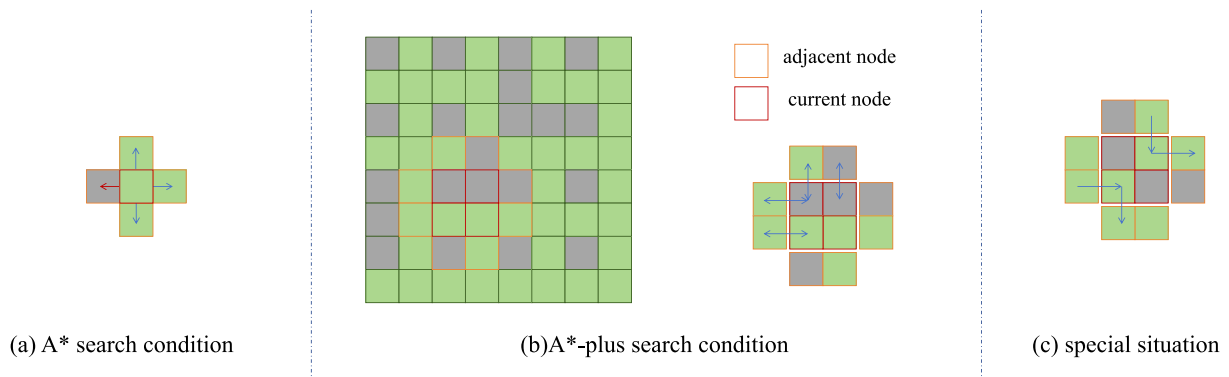


Fig. 2. Core logic of node adjacency in A*-plus.

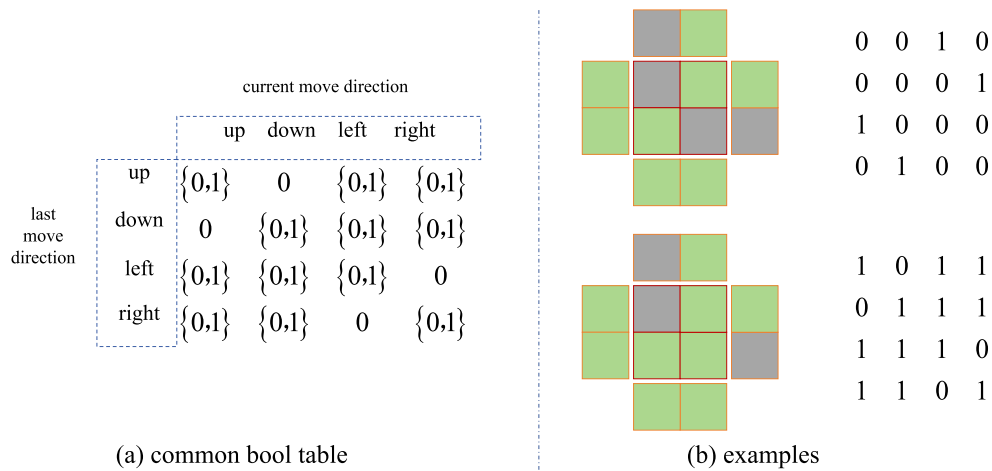


Fig. 3. (a) Boolean matrix representation of directional rules (b) Example of matrix application.

(b) is a simple example, where the matrix on the right corresponds to the situation of the node on the left.

3.2.3. Double search in diagonal cases

In some cases, such as the path shown in Fig. 4, a single compressed node may need to be traversed twice, particularly when navigating diagonally. A*-plus permits this by allowing certain 'diagonal'

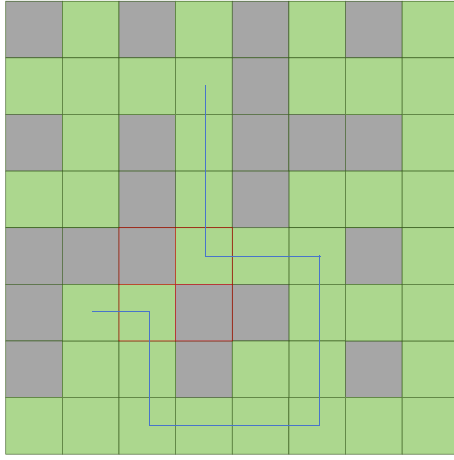


Fig. 4. Example of double search through diagonal nodes.

compressed nodes to be re-entered once during the search.

3.2.4. Directional constraints at start and goal nodes

Compressed nodes containing the start or goal point impose limitations on entry and exit directions. As shown in Fig. 5, a start node may only allow movement to the left and downward; and the goal may only be approached from the left or downward.

3.2.5. Priority queue search and heuristic optimization

Designing the open list of the A* algorithm as a priority queue is one of the most common techniques to improve search efficiency [12]. Typically, this is paired with a heuristic function to guide the search process and accelerate pathfinding [13]. Most heuristic strategies developed for traditional A* can be directly applied to A*-plus without modification.

In this work, we adopt the Manhattan distance as the heuristic function. This is because the Manhattan distance is more applicable to the 4-adjacent grid (where movement is only allowed up, down, left, and right). For each node i in the priority queue, its priority evaluation value f_i is computed as shown in Equation (1):

$$f_i = g_i + wh_i \quad (1)$$

where:

g_i is the actual cost to reach node i from the start node,

h_i is the estimated cost (heuristic) from node i to the goal node,

w is a tunable weight that adjusts the influence of the heuristic.

The actual cost g_i is the total path length accumulated to reach node i .

The heuristic cost h_i is defined using the Manhattan distance to the goal node, as shown in Equation (2):

$$h_i = |x_i - x_{goal}| + |y_i - y_{goal}| \quad (2)$$

Here, (x_i, y_i) and (x_{goal}, y_{goal}) represent the coordinates of the current node and the goal node, respectively.

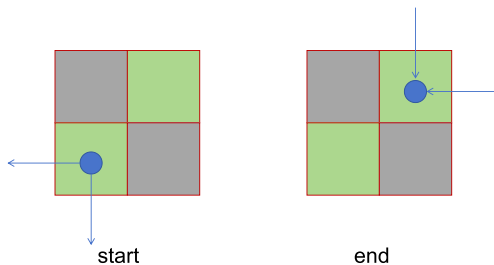


Fig. 5. Directional constraints for start and goal nodes.

3.2.6. Path backtracking method

In the standard A* algorithm, the path is reconstructed by recording the parent of each visited node and backtracking from the goal node to the start node once the search is complete. This approach yields an exact path through individual grid cells. However, in the A*-plus algorithm, the initial backtracking process only yields a coarse path between compressed nodes. To achieve a precise path at the grid-cell level, an enhanced backtracking mechanism is required.

As illustrated in Fig. 6, consider the scenario of moving horizontally from one compressed node to the next. In the straightforward case shown in Fig. 6(a), if the transition from the current node to the adjacent one is unobstructed, the algorithm directly traverses two grid cells to the right. In contrast, Fig. 6(b) depicts a case where the direct path is blocked. Here, the algorithm determines whether the current position lies in the upper or lower part of the compressed block. If in the upper section, it first moves one cell downward, then proceeds rightward. If in the lower section, it first moves upward before continuing to the right.

This approach ensures that the final reconstructed path is both valid and precise, recovering an accurate step-by-step trajectory through the original grid structure.

3.2.7. Algorithm flowchart

To clearly illustrate the modifications of the A*-plus algorithm compared to the traditional A* algorithm, we first present the flowchart of the standard A* algorithm, as shown in Fig. 7.

Fig. 8 presents the complete flowchart of the A*-plus algorithm, illustrating all components of its logic. Compared to the standard A* process shown in Fig. 1, A*-plus introduces several key modifications.

I. Preprocessing Step during Initialization:

A*-plus includes a preprocessing phase in which all map nodes are transformed into compressed nodes. This step requires additional computation and the declaration of extra variables to store the compressed node map. As a result, the initialization time of A*-plus is longer than that of the original A* algorithm. Additionally, a supplemental data structure—referred to as the Half Closed List—is introduced.

II. Handling of Visited Nodes:

When expanding the current best node, the algorithm distinguishes between regular and diagonal nodes. If the node is not a diagonal one, it is directly moved to the Closed List. If it is a diagonal node and not yet in the Half Closed List, it is added there instead. A second visit to a diagonal node triggers its transition from the Half Closed List to the Closed List.

III. Neighbor Node Evaluation:

During neighbor exploration, the allowable movement directions are constrained based on the previous movement direction of the current node. This constraint is critical to maintaining consistency in directional transitions for compressed nodes.

3.2.8. Pseudo-code

The A*-plus algorithm consists of two key components: the map preprocessing function and the A-plus path planning method*, for which the corresponding pseudocode is provided below.

First, the preprocessing function takes as input a Boolean grid map, where 0 represents an obstacle and 1 represents a traversable cell. The output is a Boolean table of compressed nodes, where each compressed node contains 4×4 Boolean values indicating the allowable movement directions given the previous movement direction. The detailed implementation is shown below.

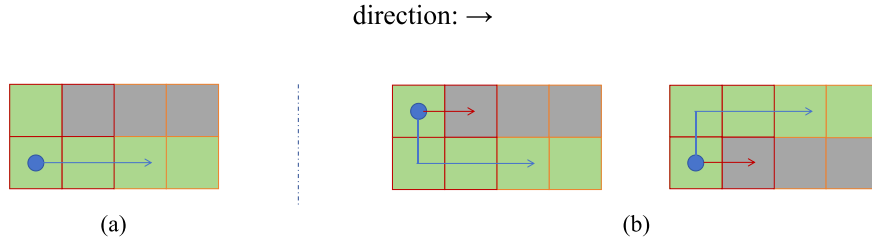


Fig. 6. Schematic diagram of the path backtracking mechanism in A*-plus.

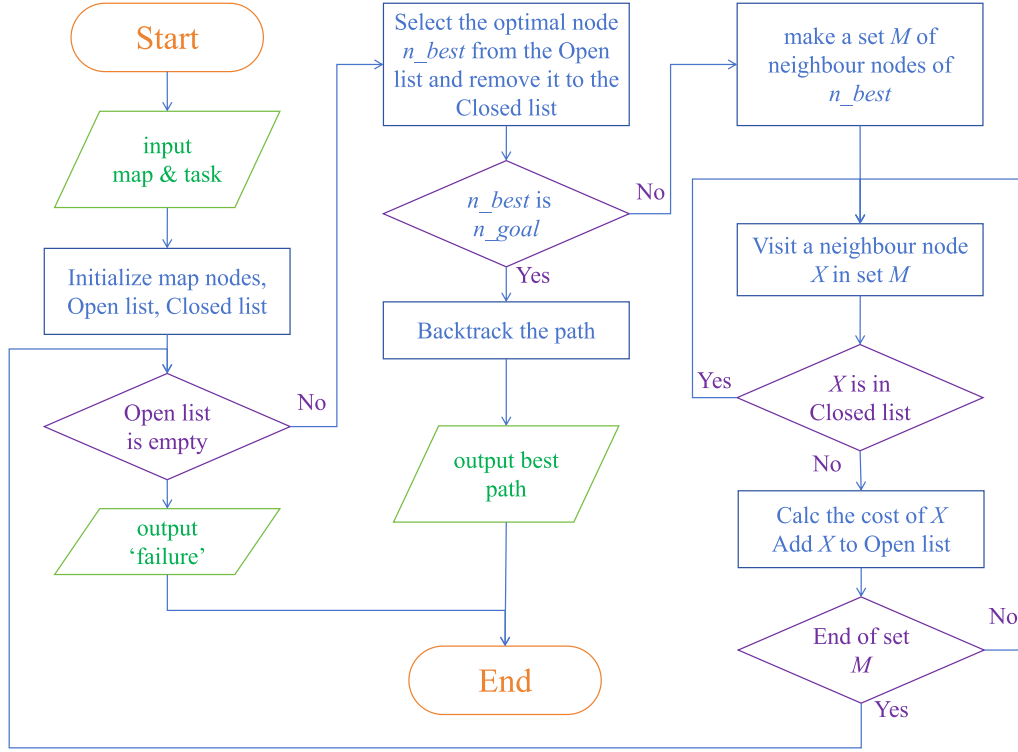


Fig. 7. A* Algorithm flowchart.

Algorithm 1

A*-plus Map Preprocessing Function

Input: Map G **Output:** Compressed map G_c

1. Initialize compressed map G_c as a 4D boolean array of size $(L/2) \times (W/2) \times 4 \times 4$, where L and W are the length and width of the original map G .
2. For $i = 1$ to $L/2$:
 - For $j = 1$ to $W/2$:
 - (a) Group four adjacent nodes $(2i-1, 2j-1)$, $(2i-1, 2j)$, $(2i, 2j-1)$, $(2i, 2j)$ into a compressed node (I, J) .
 - (b) For each direction $k = 1$ to 4 (up, down, left, right):
 - For each direction $h = 1$ to 4 (up, down, left, right):
 - If moving in direction k allows a next move in direction h :
 - Set $G_c(I, J, k, h) = \text{true}$;
 - Else:
 - Set $G_c(I, J, k, h) = \text{false}$.
 3. Return G_c .

Next, the core pseudocode of the A-plus path planning algorithm* is presented. It takes as input the original map, the compressed node map, the start coordinates, and the goal coordinates, and returns a feasible path. The implementation is detailed as follows.

Algorithm 2

A*-plus Map Path Planning Algorithm

Input: Map G , Compressed map G_c , start node s , goal node g **Output:** Path P from s to g .

1. Map the start node s to the compressed node s_c , and the goal node g to g_c . Initialize the priority queue $O = \{s_c\}$ and the closed list $C = \emptyset$.
2. While O is not empty:
 - (a) Extract the node n with the minimum $f(n) = g(n) + h(n)$ from O .
 - (b) Move n from O to C .
 - (c) If $n = g$:
 - Backtrack to obtain the compressed node path P_c ;
 - Decode P_c to map G , to obtain the final path P ;
 - Return P .
 - (d) Determine the previous moving direction k (1–4 for up, down, left, right).
 - If n is the start node, k can be arbitrary.
 - (e) For each possible next direction $h = 1$ to 4:
 - Let the location of neighbor node m be (x, y) .
 - If $G_c(x, y, k, h) = \text{true}$:
 - Update m into O .
3. Return: Path planning failed.

3.3. Theoretical analysis of optimality and efficiency

We must acknowledge that the use of the A*-plus algorithm results in

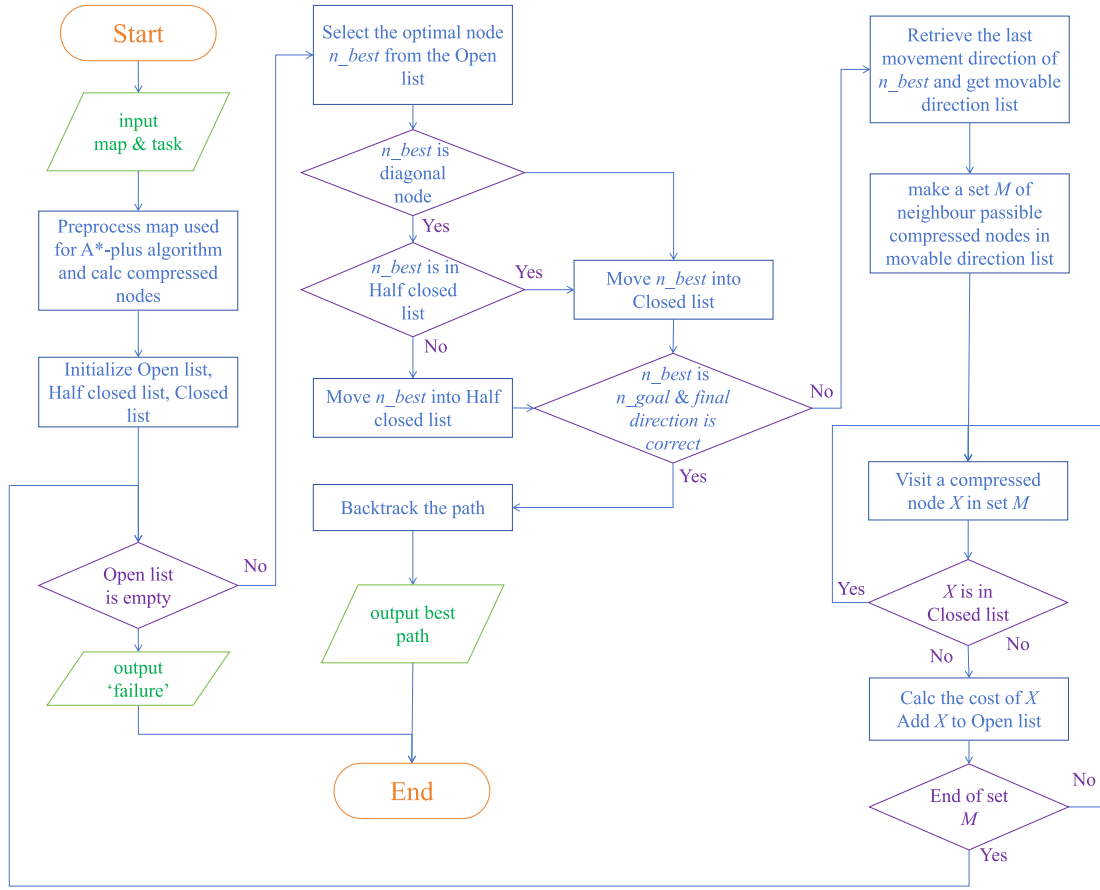


Fig. 8. A*-plus Algorithm flowchart.

a slight reduction in path optimality. This is because the compression of nodes inevitably leads to the loss of some fine-grained path information. As shown in Fig. 9 and (a) presents the optimal path generated by the traditional A* algorithm, while Fig. 9(b) and (c) show the decoded path obtained after path planning using compressed nodes in the A*-plus

algorithm.

However, the efficiency improvement achieved by A*-plus on large-scale maps is highly significant. The time complexity of the traditional A* algorithm is $O(b^d \times f(n))$, where b is the average branching factor, d is the depth of the goal node, and $f(n)$ denotes the complexity of the

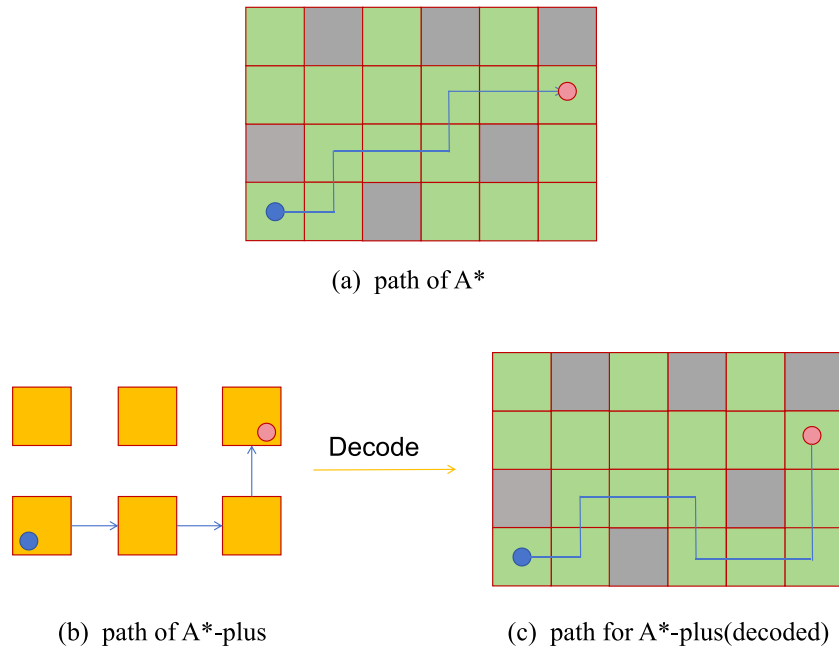


Fig. 9. The deficiency of the path quality of A*-plus algorithm.

heuristic function. Under the same rules, goals, and heuristic conditions, the A*-plus algorithm—by employing compressed nodes—reduces the time complexity to $O(b^{d/2} \times f(n))$, thus providing a substantial computational advantage on large-scale maps.

4. Experimental design and performance evaluation

4.1. Objective

The objective of this experiment is to evaluate the performance of the A*-plus algorithm in large-scale grid-based path planning. Specifically, we aim to assess its advantages over the traditional A* algorithm in terms of search efficiency, path quality, and resource consumption.

Since the improvements made by the A*-plus algorithm to the A* algorithm are at a very fundamental level and can be combined with other improvement strategies, the comparative experiments in this paper mainly focus on A* and A*-plus, as well as A* and A*-plus with the Jump Point Search strategy incorporated. Similarly, for strategies such as HPA*, the A*-plus algorithm can also be combined with these strategies.

4.2. Experimental setup

Hardware Configuration: Intel Core i7-14700 CPU, 32 GB RAM, Windows 11 OS.

Software Platform: Visual Studio 2019 with C++ (compiled using g++ 14.2.0)

Map Data: A 10,000 × 10,000 grid map generated with randomly distributed obstacles at a specific density.

4.3. Experimental plan

Compared Algorithms: A* algorithm, A*-plus algorithm, A* algorithm with JPS strategy and A*-plus algorithm with JPS strategy.

Test Scenarios: Path planning tasks conducted under identical start and goal positions across maps with obstacle densities of 5 %, 10 %, and 15 %.

Execution Method: For each configuration, 1000 random path-finding tasks are executed to obtain averaged results.

4.4. Evaluation metrics

Preprocessing Time: Total time spent reading and preprocessing the map before search.

Initial Memory Usage: Memory consumed by data structures created during preprocessing.

Total Search Time: Cumulative time spent completing all path-finding tasks.

Average Path Length: Mean length of the computed paths.

Peak Memory Usage: Maximum memory consumed during the execution of the algorithm.

4.5. Experimental results and analysis

Firstly, the legality verification. After exporting the 10,000*10,000 map with a 15 % obstacle density, the map is drawn using Matlab, and the path diagram obtained by the A*-plus algorithm is displayed, as shown in Fig. 9.

It can be seen from Fig. 10 that the path planning is effective. The enlarged figure shows that the path indeed avoids all obstacles.

Now, the path planned by the A*-plus algorithm is compared with the path planned by the A* algorithm. Similarly, we also compare the path of the A* algorithm and the A*-plus algorithm under the JPS strategy, as shown in Fig. 11.

As can be seen from the figure, there are some minor differences in the path planning of A* algorithm and A*-plus algorithm. However, for a task that cannot move diagonally but can only move horizontally or vertically, the Manhattan distances they have traversed are the very close, so the paths are **equivalent**.

The experimental results are summarized in Table 3.

4.5.1. Analysis

The first two rows of Table 1 indicate that A*-plus incurs a significantly longer preprocessing time and higher initial memory usage compared to the traditional A* algorithm. This is expected, as A* only requires loading raw grid data, whereas A*-plus must preprocess the full 100-million-node map into a compressed representation, which involves additional computation and memory allocation.

For dynamic maps, the update information must also be propagated to the compressed nodes of A*-plus, resulting in additional preprocessing overhead. However, although the preprocessing phase of A*-plus consumes approximately seven times more time than that of A*, the path planning phase of A*-plus achieves a 10–20 × improvement in efficiency compared with A*. Therefore, in tasks where path planning needs to be executed repeatedly or continuously under a largely static map, the preprocessing cost becomes amortized, leading to a significantly lower overall time consumption for the entire task.

For obstacle densities of 5 %, 10 %, and 15 %, A*-plus generates

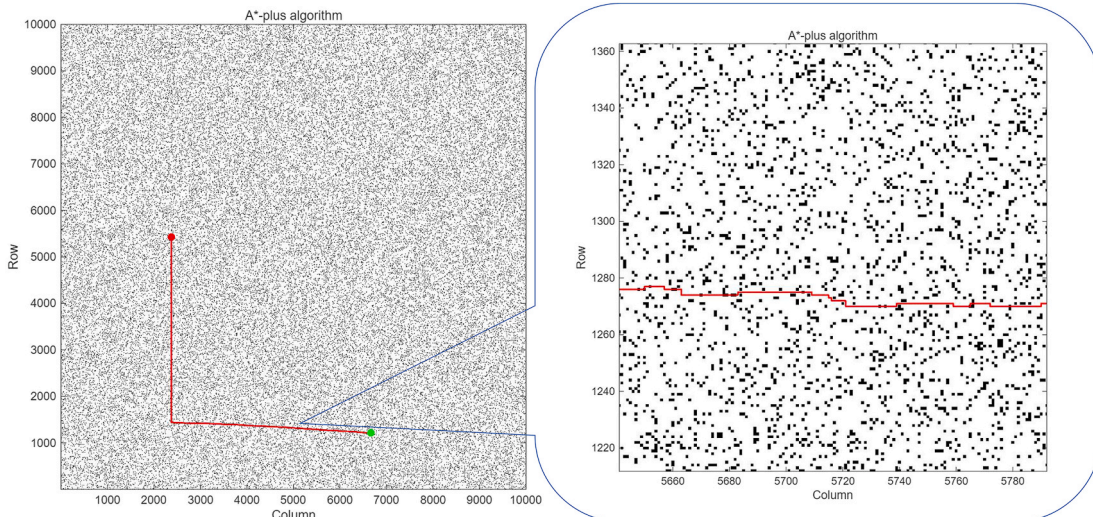
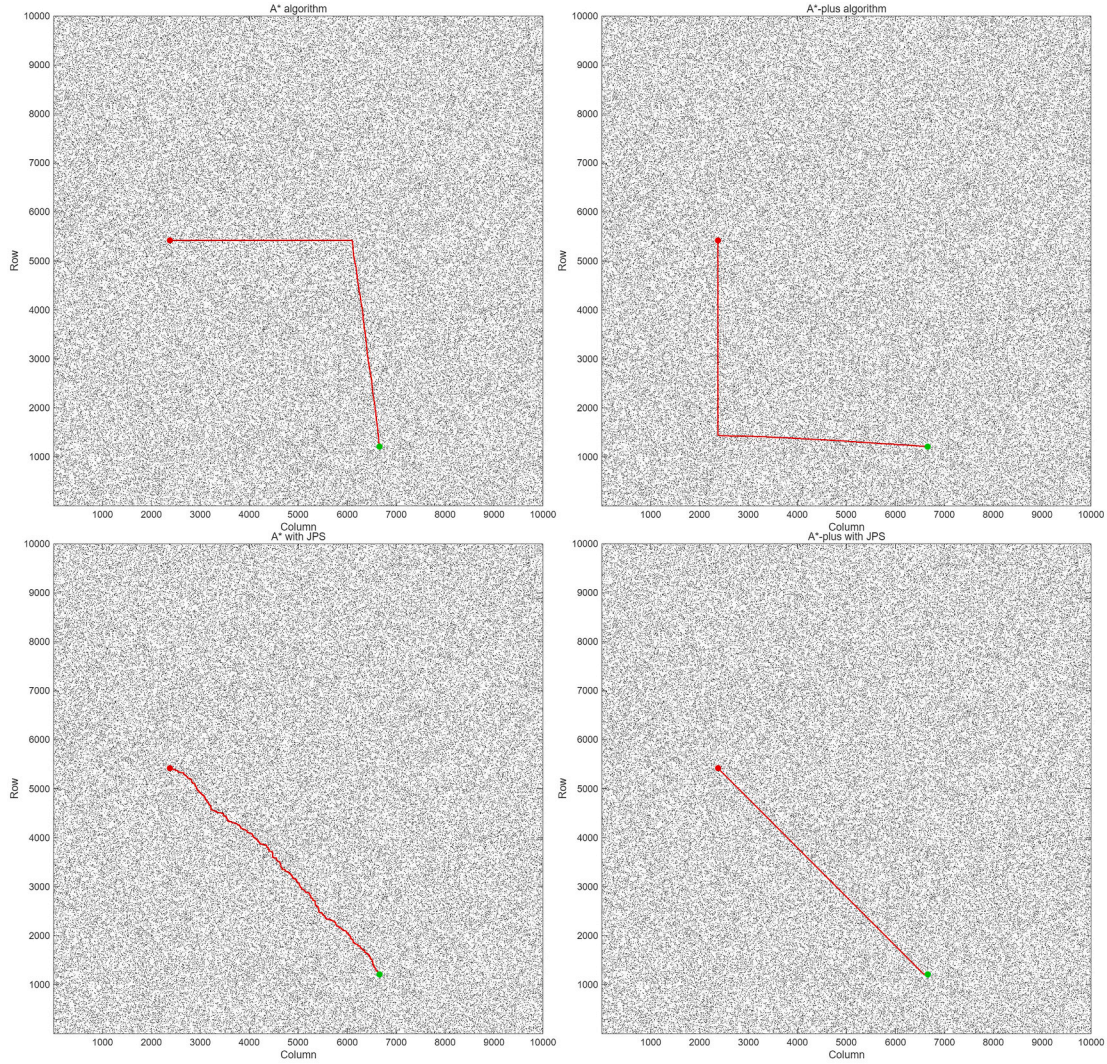


Fig. 10. Legal path from A*-plus algorithm.



Notion: In the case where movement is restricted to the four directions of up, down, left, and right, these four paths are approximately equivalent.

Fig. 11. Path comparison chart of A*, A*-plus, A*-JPS and A*-plus-JPS algorithm.

Table 3
Performance comparison between A*, A*-plus, A*-JPS and A*-plus-JPS.

Evaluation Metric	A*	A*-plus	A*-JPS	A*-plus-JPS
Preprocessing Time	40.643 s	281.818 s	–	–
Preprocessing Memory	18 MB	82 MB	–	–
Avg. Path Length (5 % obstacles)	6209.147	6213.873	6242.535	6249.802
Total Search Time (5 % obstacles)	2855.065 s	150.831 s	2042.955 s	113.436 s
Peak Memory Usage (5 % obstacles)	347 MB	135 MB	254 MB	112 MB
Avg. Path Length (10 % obstacles)	6179.053	6196.034	6280.232	6299.351
Total Search Time (10 % obstacles)	5299.612 s	345.931 s	3515.477 s	246.119 s
Peak Memory Usage (10 % obstacles)	358 MB	144 MB	256 MB	114 MB
Avg. Path Length (15 % obstacles)	6895.741	6929.412	6969.091	7021.338
Total Search Time (15 % obstacles)	7219.065 s	411.358 s	6012.043 s	353.963 s
Peak Memory Usage (15 % obstacles)	395 MB	162 MB	262 MB	114 MB

paths with lengths nearly identical to those produced by A*, confirming that it preserves path quality. More importantly, A*-plus achieves approximately a 93 % reduction in search time and 60 % lower peak memory usage across all tested scenarios. However, the paths generated by A*-plus are about 0.5 % longer than those of A*, as the use of compressed nodes slightly reduces search precision. These results demonstrate that A*-plus achieves significant computational efficiency gains while sacrificing only a minimal amount of path optimality.

Theoretical analysis suggests that this improvement mainly stems from the substantially smaller size of the priority queue. Due to node compression, A*-plus processes roughly one-fourth as many nodes as A*, reducing both the frequency and computational cost of queue operations, which leads to faster runtime and lower memory usage.

Moreover, the results indicate that under all obstacle densities, the standalone A*-plus algorithm provides better path quality and higher planning efficiency than using the JPS strategy alone. As obstacle density increases, the efficiency of JPS decreases significantly—its speedup drops from $1.4 \times$ at 5 % density to $1.2 \times$ at 15 % density, as more obstacles increase the number of jump points. Combining A*-plus with the JPS strategy further improves search speed (by approximately 33 %) but also slightly reduces path optimality, producing paths about 2 % longer than the optimal result. Regardless of obstacle density, integrating JPS

reduces memory usage by around 30 %.

Overall, these findings confirm the effectiveness and efficiency of the A*-plus algorithm for large-scale grid map path planning. In practical engineering applications, A*-plus is well suited for scenarios requiring repeated path planning tasks, where it achieves substantial efficiency gains with only minimal loss in path quality. For cases demanding even higher efficiency and where slight degradation in optimality is acceptable, a combined approach integrating A*-plus with JPS or other improvement strategies can provide an advanced and flexible solution.

5. Results and discussion

5.1. Summary of key findings

The proposed A*-plus algorithm demonstrates significant advantages in solving large-scale grid-based path planning problems. Compared to the traditional A* algorithm, A*-plus improves search efficiency by approximately 15–20 times, while maintaining path optimality. Additionally, the number of expanded nodes and overall memory consumption are substantially reduced, indicating better utilization of computational resources.

5.2. Comparison with existing research

When compared to other optimized algorithms such as Jump Point Search (JPS) and Hierarchical Path-Finding A* (HPA*), A*-plus shows superior performance on large-scale maps. JPS improves efficiency by skipping redundant nodes but is less effective in complex obstacle environments. HPA* reduces search space through hierarchical abstraction, but the transition between layers may introduce path inaccuracies.

In contrast, A*-plus integrates multiple strategies—multi-level map abstraction, heuristic optimization, and directional search pruning—into a unified framework. This combination allows A*-plus to maintain path optimality while significantly accelerating the search process, making it more suitable for high-performance path planning in expansive environments.

Recent intelligent cooperative approaches employ deep learning and model aggregation techniques to achieve adaptive decision-making in dynamic UAV environments [18]. In contrast, the proposed A*-plus algorithm focuses on deterministic path search optimization, emphasizing computational efficiency and structural improvement rather than data-driven learning. While AI-based cooperative frameworks provide strong adaptability, they require extensive training and high computing resources. A*-plus, being lightweight, interpretable, and resource-efficient, is more suitable for large-scale or static path planning tasks. These two directions are complementary, and future work may explore integrating A*-plus with intelligent cooperative frameworks to enhance adaptability in dynamic scenarios.

5.3. Interpretation and theoretical analysis

The performance improvement of the A*-plus algorithm is primarily attributed to its node compression mechanism. By aggregating each 2×2 group of grid cells into a single node, the effective map size is reduced to one-fourth of the original, thereby lowering the dimensionality of the search space. This significantly reduces the size of the Open List, which in turn minimizes memory usage during execution.

Moreover, since the Open List is implemented as a priority queue, a smaller number of nodes results in fewer comparisons when inserting new nodes. This reduction in comparison operations directly enhances the computational efficiency of the algorithm.

5.4. Limitations and future work

Despite the promising results achieved by A*-plus in large-scale grid-based path planning, several limitations remain.

I. Loss of certain global optimization:

Although this compression node search method can roughly find the optimal path, it will lose certain globality within the compressed nodes. As shown in Fig. 12, when the general trend is to the right and downward, the path planned by the A*plus algorithm still shows some upward movement situations. This is because, when the A*plus algorithm performs the path decomposition of "moving one compressed node to the right" during the path backtracking, it will prioritize "moving to the right" and may ignore the global downward movement requirement.

II. Adaptability to Dynamic Environments:

The current algorithm is designed for static maps and lacks mechanisms to handle dynamic changes, such as moving obstacles.

III. Three-Dimensional Path Planning:

This study focuses exclusively on two-dimensional grid maps. The algorithm has not yet been extended to support 3D environments, which limits its applicability to certain scenarios like aerial navigation.

IV. Validation in Real-World Systems:

The experiments are currently limited to simulated environments. Real-world validation on physical robots or autonomous navigation systems has not yet been conducted.

Future research may consider the following directions.

I. Take into account the overall trend of the path:

When considering conducting local path planning within the compression node, based on the global path trend, actively adjust the path direction.

II. Path Planning in Dynamic Environments:

For dynamic environments, it is not necessary to rebuild the entire compressed map when local changes occur. Instead, only the affected regions can be updated by synchronizing the modified parts of the original map with their corresponding compressed nodes in the A*-plus structure. Future research may further investigate the local propagation effects of such updates.

III. Extension to 3D Path Planning:

The compression and search principles of A*-plus can be generalized to three-dimensional voxel grids. Each 3D compression node would represent a volumetric region rather than a 2D block, and heuristic estimation could leverage 3D Euclidean or octile distance metrics. Given the computational scalability demonstrated in 2D, extending A*-plus to 3D environments (e.g., UAV or underground navigation) is conceptually feasible and will be investigated in future work.

IV. System Integration and Field Testing:

Integrate A*-plus into actual robotic platforms or autonomous vehicles and evaluate its performance under real-world conditions.

Additionally, since A*-plus is fundamentally a variant of the A* algorithm, it remains theoretically compatible with most existing A* optimization strategies. Future work could explore the combination of A*-plus with other heuristic enhancements, pruning techniques, or data structures to further boost its performance on large-scale maps.

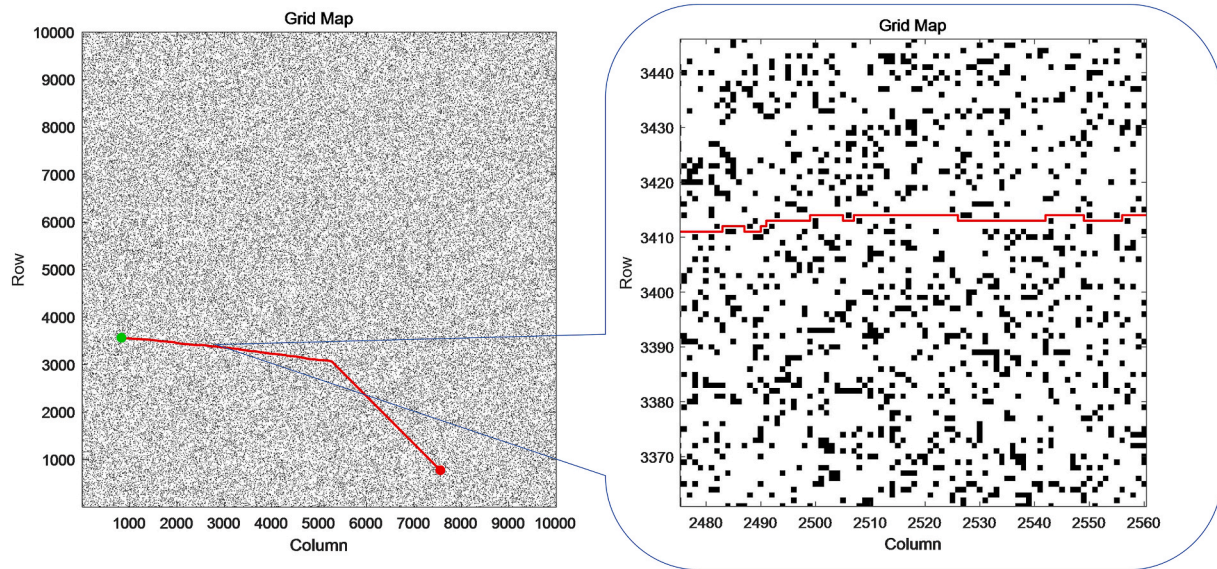


Fig. 12. Global optimization limitation of A*-plus algorithm.

CRedit authorship contribution statement

Chaosheng Zhang: Writing – original draft, Visualization, Project administration, Methodology, Funding acquisition, Formal analysis, Conceptualization. **Qingsen Hu:** Writing – review & editing, Investigation, Data curation. **Jiangang Tu:** Writing – review & editing, Supervision, Funding acquisition. **Yu Dong:** Resources, Data curation. **Yizhong Tan:** Data curation, Conceptualization. **Yufeng He:** Visualization, Data curation.

Declaration of competing interest

All the authors of article ‘A-plus Path Planning Algorithm for Large-Scale Grid Maps’ declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Supplementary data

Supplementary data to this article can be found online at <https://doi.org/10.1016/j.array.2025.100593>.

Data availability

Data will be made available on request.

References

- [1] Harabor Daniel, Grastien Alban. Improving jump point search. *Proceedings of the international conference on automated planning and scheduling*, vol. 24; 2014.
- [2] Nannicini Giacomo, et al. Bidirectional A* search on time-dependent road networks. *Networks* 2012;59(2):240–51.
- [3] Kui Luchao, Yu Xianwen. A pathfinding algorithm for large-scale complex terrain environments in the field. *ISPRS Int J Geoinf* 2024;13(7):251.
- [4] Peque Jr G, Urata J, Iryo T. Preprocessing parallelization for the ALT-Algorithm. *International conference on computational science*. Cham: Springer International Publishing; 2018. p. 89–101.
- [5] Du Wei, Islam Fahad, Likhachev Maxim. Multi-resolution A*. *Proceedings of the international symposium on combinatorial search*, vol. 11; 2020. No. 1.
- [6] Ebendt Rüdiger, Drechsler Rolf. Weighted A* search—unifying view and application. *Artif Intell* 2009;173(14):1310–42.
- [7] Hart Peter E, Nilsson Nils J, Raphael Bertram. A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans Syst Sci Cybern* 1968;4(2): 100–7.
- [8] Hong Zhonghua, et al. Improved A-Star algorithm for long-distance off-road path planning using terrain data map. *ISPRS Int J Geoinf* 2021;10(11):785.
- [9] Foad Daniel, et al. A systematic literature review of A* pathfinding. *Procedia Comput Sci* 2021;179:507–14.
- [10] Wang Huanwei, et al. The EBS-A* algorithm: an improved A* algorithm for path planning. *PLoS One* 2022;17(2):e0263841.
- [11] Ou Yangqi, et al. Improved A* path planning method based on the grid map. *Sensors* 2022;22(16):6198.
- [12] Yodianto Wilsen, et al. Searching routing using A-Star (A*) search algorithm. 2024 3rd international conference on creative communication and innovative technology (ICCIIT). IEEE; 2024.
- [13] Zhang Jing, et al. Autonomous land vehicle path planning algorithm based on improved heuristic function of A-Star. *Int J Adv Rob Syst* 2021;18(5): 17298814211042730.
- [14] Li Yibo, Dong Dingguang, Guo Xiaonan. Mobile robot path planning based on improved genetic algorithm with A-star heuristic method. 2020 IEEE 9th joint international information technology and artificial intelligence conference (ITAIC), vol. 9. IEEE; 2020.
- [15] Zhang Lin, Zhang Yingjie, Li Yangfan. Mobile robot path planning based on improved localized particle swarm optimization. *IEEE Sens J* 2020;21(5):6962–72.
- [16] Lan Xing, et al. Research on robot global path planning based on improved A-star ant colony algorithm. In: 2021 IEEE 5th advanced information technology, electronic and automation control conference (IAEAC). vol. 5. IEEE; 2021.
- [17] Gao Jianfeng, et al. Fast path planning for firefighting UAV based on A-Star algorithm. *J Phys Conf* 2021;2029(1). IOP Publishing.
- [18] Zhou L, et al. Cooperative generative AI for UAV-based scenarios: an intelligent cooperative framework. *IEEE Veh Technol Mag* June 2025;20(2):44–52. <https://doi.org/10.1109/MVT.2025.3544817>.